

Document ingestion

LESSON 11 · MODULE 3 — KNOWLEDGE & MEMORY

What you'll learn

- How to ingest real files (PDF, DOCX, websites) into a knowledge base — not hard-coded strings
- The two chunking parameters worth knowing about: `chunk_size` and `chunk_overlap`, and sensible defaults
- The three PDF parsers Lyzr offers (`llmsherpa`, `pymupdf`, `laserparse`) and when to reach for each
- How the same retrieve-then-shape pattern from lesson 10 holds when the documents get real

Key concepts

Same shape as lesson 10, different doc. Create a knowledge base.

`add_pdf` the file. Run the query. The agent quotes from the file.

TAKEAWAY

Same shape as lesson 10, different doc.

Two chunking parameters worth knowing. `chunk_size` — how much text goes into each searchable chunk, measured in tokens. Default 1024. Bigger chunks → more context per retrieval, lower precision.

Smaller chunks → more precise retrieval, more chunks to score.

`chunk_overlap` — how much consecutive chunks share. Default 128.

Some overlap is good — it stops you from cutting an important sentence in half across two chunks. **Don't tune these unless you have a reason.**

TAKEAWAY

Two chunking parameters worth knowing.

Three PDF parsers, three different jobs.

TAKEAWAY

Three PDF parsers, three different jobs.

- `llmsherpa` (default) — layout-aware. Reads the visual structure of the PDF; preserves headings, tables, bullet hierarchies. **Best for structured docs:** whitepapers, manuals, reports.
- `pymupdf` — fast, text-only. No layout awareness, minimal overhead. **Reach for this** when you've got clean text-heavy docs (legal contracts, plain prose) and don't need visual structure.
- `laserparse` — multimodal. Uses DocLin plus a vision model under the hood. **For visually complex documents** — technical drawings, slide decks, anything where images and charts carry information that text alone can't capture. Slower and more expensive, but it sees what others can't.

Picking rule. Layout-aware default for most things. Switch to PyMuPDF for speed on plain text. Reach for laserparse when the visuals matter.

TAKEAWAY

Picking rule.

The retrieval pattern is unchanged from lesson 10. Same strict-refusal instructions. Same `knowledge_bases=[kb]` at runtime. The model adds shape (numbered steps, formatted lists, footnotes) on top of grounded retrieved facts.

TAKEAWAY

The retrieval pattern is unchanged from lesson 10.

Code shown in this lesson

PYTHON

```
# Setup includes fpdf2 to generate a sample PDF for the demo;
# in production you'd skip this and point at a real file.

kb = studio.create_knowledge_base(name="veristack-handbook")

# Default parser (llmsherpa), default chunking
kb.add_pdf("veristack_admin_handbook.pdf")

agent = studio.create_agent(
    name="veristack-support",
    provider="gpt-4o",
    role="VeriStack customer support agent",
    goal="answer admin questions from documentation only",
    instructions="Only answer from documentation. Refuse if not in docs.",
)

response = agent.run(
    "How do I export workspace data?",
    knowledge_bases=[kb],
)
print(response.response)
```

PYTHON

```
# Picking a parser explicitly
kb.add_pdf("structured_whitepaper.pdf", data_parser="llmsherpa") # default – layout-aware
kb.add_pdf("legal_contract.pdf", data_parser="pymupdf") # fast, text-only
kb.add_pdf("technical_drawings.pdf", data_parser="laserparse") # multimodal vision
```

PYTHON

```
# Tuning chunk parameters (don't unless you have a reason)
kb.add_pdf("large_report.pdf", chunk_size=1024, chunk_overlap=128)
```

TRY THIS

- 1 Ingest the same PDF three times into three separate knowledge bases — `chunk_size=512`, `1024`, and `2048`. Ask the same question against each. Note where retrieval precision changes and where it doesn't.
- 2 Pick a visually-rich PDF you have (a slide deck, a product spec with diagrams). Ingest it three times with `data_parser="llmsherpa"`, `"pymupdf"`, and `"laserparse"`. Ask *"what's on page 5?"* against each. Compare how well each parser captured the visual content.